

42891 Infrastructure for Cloud Computing

Assignment 1 — Spring 2026

Field	Detail
Student name	Jinkyung Kim
Student ID	14657314
Subject	42891 Infrastructure for Cloud Computing

Question 1

1.1 What is a cloud?

A cloud is a pool of virtualised computer resources (Zhang 2026a). The pool carries many unrelated workloads at once, and it hands out hardware, software and data sets dynamically, as they are asked for.

What makes something a cloud rather than a well-run server room is not the equipment. It comes down to three properties:

- **Pooling.** Physical resources are aggregated and abstracted, so a consumer is served out of a shared pool instead of from an identified machine (Mell & Grance 2011).
- **Multi-workload hosting.** The same pool simultaneously carries unrelated and heterogeneous workloads belonging to different tenants.
- **On-demand provisioning.** Capacity is allocated and released programmatically, in minutes, rather than through a procurement cycle.

Finally one terminology point that should be nailed down. Because these two senses of the word are constantly confused: As a noun, cloud refers both to the resource pool and also to the delivery infrastructure in front of that pool. Cloud computing refers to the consumption model of that pool (Erl, Puttini & Mahmood 2013). You can virtualise your whole data centre and it still does not count as cloud computing if what emerges at the other end is never offered as a metered, self-service resource..

1.2 What is cloud computing? (1 point)

The standard definition is the NIST one:

"Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction" (Mell & Grance 2011, p. 2).

Starting from that point NIST divides the model into five essential characteristics — on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service — then three service models and four deployment models (Mell & Grance 2011). Erl, Puttini and Mahmood (2013, p. 27) come at it from the other direction. They define cloud computing as "a specialized form of distributed computing that introduces utilization models for remotely provisioning scalable and measurable resources". NIST explains

what the consumer experiences. Erl explains the machinery that creates that experience. Both definitions happen to land on the idea of measurement. That is not an accident. Without metering you have no pay-per-use, and without pay-per-use pricing the cost model below cannot exist.

1.3 The cloud cost model

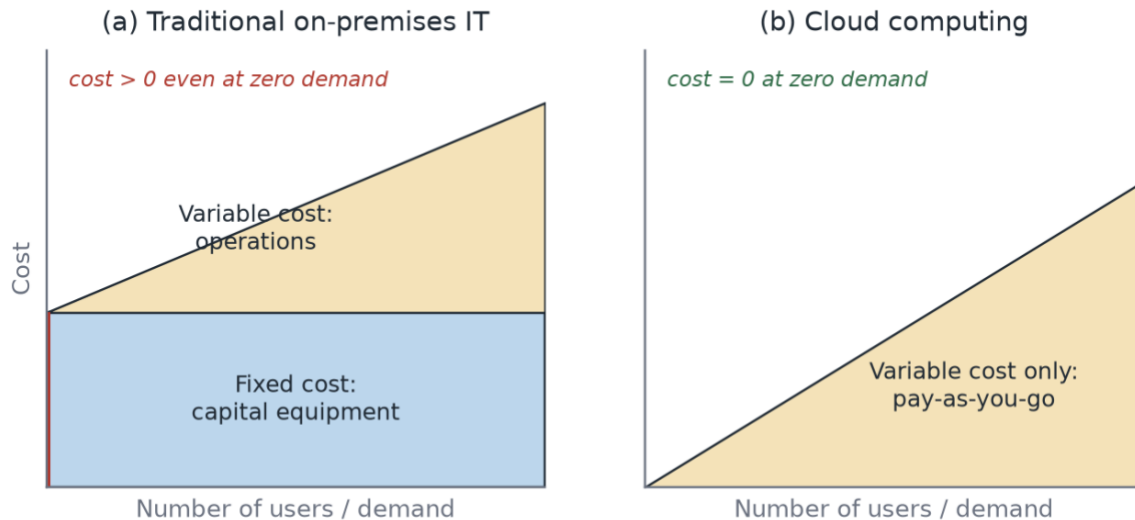


Figure 1. The cloud cost model. Traditional IT carries a fixed capital cost incurred before any demand arrives; cloud computing turns that block into a variable operational cost proportional to consumption. Adapted from Zhang (2026a).

The issue here is that these two curves are different in shape, not just height. That is literally all that is at stake.

Under the traditional model organisation purchases and owns hardware, system software, and applications sized to meet peak demand, then installs/configures/ tests-verifies-evaluates/ manages everything (Zhang 20 26a). The outcome is significant upfront fixed cost before the first user arrives and then layers of variable cost on top of that. With cloud computing the consumer simply subscribes and pays as used against a defined quality of service. There is no big fixed block.

Three observations from that statement alone. Those are why the model is significant commercially and not just mathematically.

- **CapEx is converted to OpEx.** Spending goes off the balance sheet and into the operating budget. That dramatically lowers the barrier to starting anything. It also removes the write-off risk of having bought the wrong amount of capacity.
- **The provisioning gamble is shifted to the provider.** As Armbrust et al. (2010) put it: Fixed capacity must be sized either for the peak (wasting most of it most of the time), or below the peak, in which case the demand you turn away is lost revenue that can easily exceed whatever you saved on hardware. Elasticity eliminates the need to gamble either way.
- **Economies of scale are shared.** Providers purchase power/hardware/bandwidth at scales no single enterprise can hope to match. Savings from that and other factors over buy-and-own are put in the range of 80–95% for workloads that are suitable for the cloud (Zhang 2026a).

None of which means cloud is automatically cheaper. A workload that consistently and maximally utilises its resources all day every day, for several years has nothing to sell back and eventually paying an ongoing margin on capacity that could just be owned outright will be more expensive. Marston et al. (2011) pose the decision as being between demand variability and time horizon rather than unit cost, which I think cuts to the heart of the matter: the cost model is only of value when demand is unpredictable, spiky or transient, and irrelevant when demand is steady and expected.

Question 2

2.1 The cloud service models

NIST defines three, separated by how much of the technology stack the provider operates on your behalf (Mell & Grance 2011):

- Infrastructure as a Service (IaaS) — the provider offers processing, storage, networks, and other fundamental computing resources. The consumer can deploy and run arbitrary software, which can include operating systems, on those resources.
- Platform as a Service (PaaS) — the consumer can deploy onto the provider's cloud environment consumer-created or -acquired applications written in languages supported by the provider. The consumer does not manage the underlying servers, operating systems, or storage infrastructure.
- Software as a Service (SaaS) — the consumer uses applications provided by the supplier who runs them on cloud infrastructure. The consumer does not control the underlying cloud infrastructure, including network, servers, operating systems, or storage, and has limited control over the application's configuration settings.

One mistake often made is to present them in the same list as public, private, community and hybrid. Those are deployment models and belong on a different axis (Mell & Grance 2011; Zhang 2026a). Any service model can be provided in any deployment model, so private PaaS and public PaaS are both entirely mundane and the two lists should not be conflated. ISO/IEC (2014) also defines service categories such as Communications as a Service, Network as a Service and Data Storage as a Service. These are cousins of the three original triumvirate, not additional levels on the tower; Network as a Service is Example 7.

2.2 The three models and the relationships among them

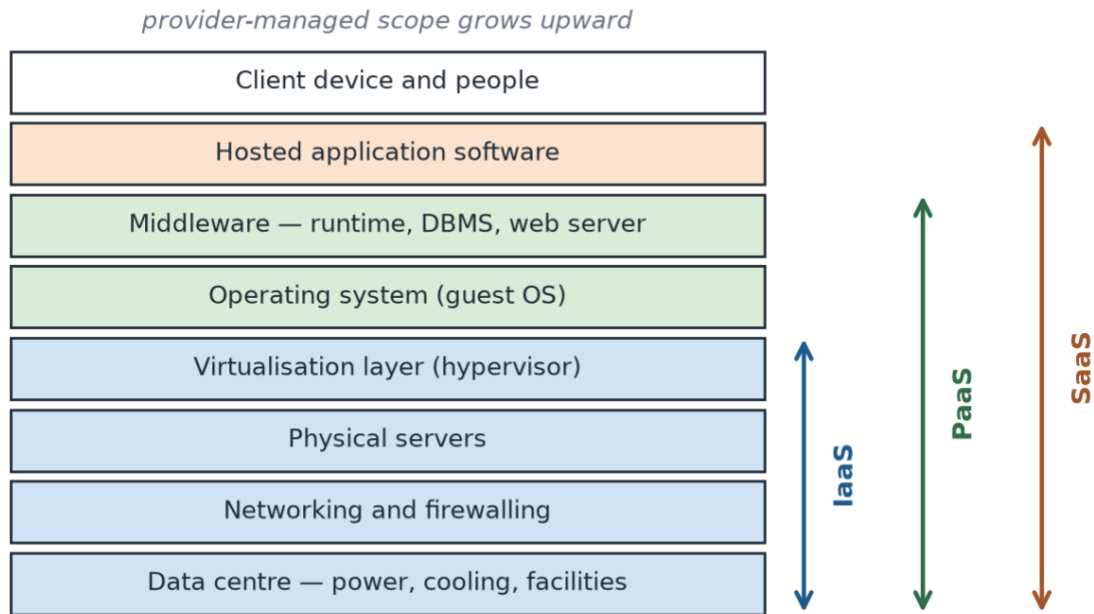


Figure 2. Service-layer view of IaaS, PaaS and SaaS. Each bracket shows how far up the stack the provider's responsibility reaches; everything above the bracket stays with the consumer. Adapted from Zhang (2026a).

Figure 2 answers the 'relationships' half of this question more directly than prose can. All three models describe the same stack of layers. They differ only in where the line gets drawn.

Infrastructure as a Service

IaaS provides you with a virtual server, virtual storage and virtual networking, and then steps out of the way. Starting at the guest operating system and upward you own everything: patches, middleware, runtime, code, data. The service contract is written in terms of the infrastructure. The previously referenced article provides an example contract – A virtual server configured with 32 GB of RAM and 4 GB of local storage. 99.5% SLA with no failover. Costs are \$0.95/hour plus \$.05 per GB of outbound data transfer (Zhang 2026b). Maximum flexibility. Maximum amount of operational overhead. Amazon EC2, Azure Virtual Machines and Google Compute Engine are the poster children.

Platform as a Service

PaaS gives you a ready-to-use environment. Operating system, runtime, application server, typically a managed database: these are already installed and maintained. All that the consumer provides is an application and its configuration and nothing else.

Notice how Erl's picture of PaaS puts question marks below the line. That question mark is the interesting part of the picture. It signifies that the consumer is blissfully ignorant of the implementation (Zhang 2026b). Normally you cannot know on which physical host your code is running or even how many. That's not an inconvenience the provider sorry about; that's what allows them to patch and scale and move things around under a running application. You can buy a service level agreement for an application server plus

DBMS platform at 99.5% availability with automatic scaling for \$0.45/hour per 500 million requests (Zhang 2026b). AWS Elastic Beanstalk, Microsoft Azure App Service and Google App Engine are examples of PaaS providers.

Software as a Service

SaaS delivers a complete application. The user consumes the published interface and the agreement, with zero visibility into IT resources or implementation details (Zhang 20 26b). Control is reduced to tenant configuration: users, roles, branding, workflow options. Notice also that the SLA uses different language—the 0.5 ms response time at \$0.05 per 100 requests describes the application, not servers. Salesforce, Microsoft 365, Google Workspace.

How the three relate

1. Four relationships apply here. Any answer addressing only the first relationship is incomplete.
2. 1. They stack. Each model consists of the layers below it: PaaS sits on infrastructure equivalent to IaaS, SaaS on a platform equivalent to PaaS. Provisioning of physical and virtualised resources (IaaS) underpins all of the layers above it (Zhang 20:26a).
3. 2. Control and management burden vary inversely. Moving up the stack you give up control and are freed from operations. Down the stack you assume that control and all of the work that comes with it. There's no such thing as a better model - only different ones. The question is at what layer of the stack an organisation wishes to operate when something breaks on Tuesday afternoon.
4. 3. Abstraction grows with each upward step. This is what allows the provider to change, grow and repair whatever's beneath that layer without consumer knowledge. It's also the root of the weakness that comes with each upward layer - what you don't control you can't tune, audit directly, or easily move to another provider.
5. 4. The dependencies are logical, not vendor relationships. The stack simply defines levels of responsibility. A SaaS provider could be utilizing somebody else's IaaS, or they could own their own data centers and expose nothing but the application. Customers can (and often do) mix models as well — consuming SaaS for email, PaaS for a customer portal, and IaaS instances hosting a legacy line-of-business application that simply will not migrate.

Table 2. Responsibility boundaries and trade-offs across the three service models.

Aspect	IaaS	PaaS	SaaS
Consumer controls	Guest OS, middleware, runtime, application, data	Application and its configuration, application data	User-level configuration only
Provider manages	Virtualisation, servers, storage, networking, facility	All of IaaS plus OS, runtime, middleware, scaling	The entire stack including the application
Unit of delivery	Virtual machine, volume, virtual network	Ready-made execution environment	Working application or API
SLA expressed as	Availability and capacity of the instance	Availability plus platform behaviour such as auto-scaling	Application response time and availability
Main benefit	Maximum control and workload portability	Fastest path from code to production	No operations burden at all
Main limitation	You are still running an operating system estate	Constrained to supported languages and services	Least configurable, strongest lock-in

Question 3

3.1 The cloud components

Looking at the infrastructure that actually serves workloads, a typical cloud is assembled from five components (Zhang 2026b).

Table 3. The five infrastructure components of a typical cloud (after Zhang 2026b).

Component	Role in the cloud
Virtual machines	Isolated instances of a guest operating system and its applications that behave like separate computers. Each one encapsulates its virtual hardware, virtual disks and metadata, which is what makes it copyable and movable.
Hypervisor	The virtualisation layer that creates the VMs, allocates physical resources to them, keeps them isolated from one another and takes part in moving them between hosts.
Physical servers	The compute substrate, usually commodity x86 rack or blade servers, aggregated into clusters so capacity is addressed as a pool instead of as individual machines.
Networks	Physical switching and routing, plus the virtual switches and port groups connecting VMs to each other, to storage and to the Internet.
SAN storage	Shared block-level storage reached over a dedicated fabric, letting several hosts see the same datastore. This turns out to be the precondition for live migration and high availability.

There is a second, broader list that details what technology innovations a provider must cobble together before it can offer cloud services at all: broadband networks and Internet architecture, data centre technology, virtualisation technology, web technology, multitenant technology and service technology (Zhang 2026c; Erl, Puttini & Mahmood 2013). The two lists perform different functions. The first details what physically resides in the rack; the second identifies the technology families that make the rack marketable as a service..

3.2 The cloud architecture built from these components

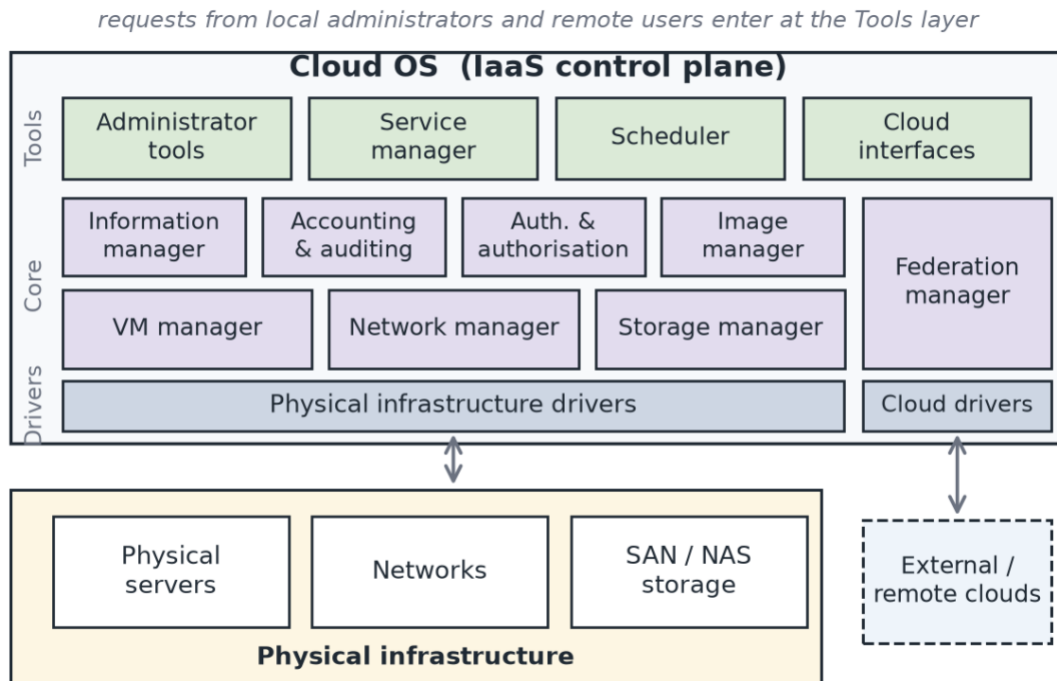


Figure 3. IaaS cloud architecture. The Cloud OS sits between users and the physical infrastructure, organised into a Tools layer, a Core layer of managers, and a Drivers layer abstracting specific hardware and remote clouds. Adapted from Zhang (2026a).

Taken individually, elements from Section 3.1 have no cloud-like behavior. What makes them a cloud is management software -- a "Cloud OS" -- that treats them as a pool.

This analogy to a conventional operating system works better than most. The OS also schedules CPU, memory, disks and I/O. The OS also provides a secure, sandboxed, multi-tenant environment in which to run user code. The OS also shields the user from the details of the underlying hardware, presenting interfaces and APIs instead (Zhang 20 26a). The difference is the granularity of what's being scheduled: a virtual machine across a cluster rather than a thread across a CPU.

Figure 3 organises the architecture into four horizontal layers.

Physical infrastructure layer

Servers, networks and storage arrays - the raw compute, networking and storage capacity from Section 3.1. Nothing here is cloud-specific, and that is intentional. The abstraction layers above are designed to make commodity hardware fungible.

Drivers layer

Physical Infrastructure drivers convert the Core layer's generic instructions into vendor specific operations against a particular hypervisor, switch or storage array. Cloud drivers perform the same function for external clouds, enabling the local installation to treat a remote provider as just another source of capacity. This layer provides the platform with its portability. Changing storage vendors updates a driver, not the managers built on top of it.

Core layer

Eight managers implement the actual cloud functions (Zhang 2026a).

Table 4. Core-layer managers of the Cloud OS and their responsibilities.

Manager	Responsibility
VM manager	Manages a VM's entire life cycle and performs VM actions — deploy, migrate, suspend, resume, shut down — according to user demand or scheduling strategy.
Network manager	Manages the private networks interconnecting a service's internal components, manages public IP address pools, and connects front-end components to the Internet.
Storage manager	Provides storage services and end-user virtual storage systems as a commodity.
Image manager	Handles a large population of VM images belonging to different users, with different operating systems and software configurations.
Information manager	Monitors and gathers state information about VMs, physical servers, network devices and storage systems.
Accounting and auditing	Obtains and records resource-usage information for deployed services. This is the metering that makes 'measured service' possible.
Authentication and authorisation	Authenticates users and administrators and gives them access only to authorised resources.
Federation manager	Enables access to remote cloud infrastructures, which is what supports hybrid and cloud-bursting patterns.

Tools layer

Administrator tools include both user management and physical- infrastructure management capabilities. Service manager involves management and support for virtualised multi-tier services. Cloud interfaces deal with exposing the provider's APIs. As most cloud products and providers supply their own API instead of using one common/shared API (Zhang 2026a), this is where vendor lock-in is focused.

The scheduler deserves separate treatment because it works at three levels that are easy to run together:

- **Physical host level**, run by the hypervisor scheduler: deciding when VMs can obtain system resources, and which physical CPUs are assigned to each VM.
- **Cloud level**, run by the Cloud OS scheduler: deciding the particular physical server where each VM is deployed.
- **Federation level**: deciding to deploy a VM in a remote cloud when there are not enough resources available locally.

Following a single request through

Examining the lifecycle of a single request makes the interplay between layers clearer than a description does. A consumer sends a request for a virtual server to the cloud interface. Authentication and authorisation verify who is making the request and if they have permission to do so. Scheduler selects an appropriate physical host that has enough capacity or offloads the request to the federation manager if none are available locally. Image manager fetches the requested VM image, storage manager provisions a volume from the shared storage pool and network manager connects the machine to the appropriate private network and provisions a public IP if necessary. Finally the VM manager tells the correct physical

infrastructure driver to instruct the hypervisor present on the selected host to actually create the machine. It is then up to the information manager to monitor its state and the accounting subsystem to meter it for billing purposes. Components introduced in Section 3.1 are all used, and each is only ever accessed through the manager responsible for it – this modularity is exactly what allows us to swap out the underlying hardware.

Zooming in further, Erl, Puttini and Mahmood (2013) detail the reusable mechanisms which bring these functions to life: logical network perimeter, virtual server, cloud storage device, cloud usage monitor, resource replication and ready-made environment (infrastructure side) and remote administration, resource management, SLA management and billing management (management side) (Zhang 2026c). Bundle all this together and you have the basic building blocks of cloud architectures – workload distribution, resource pooling, dynamic scalability and service load balancing.

Question 4

4.1 What is virtualisation?

Virtualisation abstracts computer resources: somewhere between consumer and hardware sits a software layer that presents logical resources to users, hiding, aggregating or subdividing physical ones below. It seeks to share IT resources as widely as possible, extracting maximum utilisation and reducing cost (Zhang 2026b). Architecturally it dismantles monopoly: instead of a single operating system consuming all hardware, a new layer says 'first, many operating systems must share' (Zhang 2026b).

The classic reference is Popek and Goldberg (1974), who define precisely what virtual machine monitors must provide: equivalence (programs under the VMM behave as they would on bare metal), resource control (the VMM always retains complete control of resources) and efficiency (the set of instructions that run directly on processor, without VMM intervention is a 'dominant subset'). Their criteria now purr behind most modern designs. x86 wasn't virtualisable, by their definition, so the early mitigation strategies were binary translation and paravirtualisation (Barham et al. 2003) until Intel VT-x and AMD-V added CPU support for efficient virtualisation. Today it's commonplace.

The business driver is consolidation. Physical servers devoted to one application tend to idle. Consolidate (virtualise) several such servers onto one physical host and you improve utilisation and can reduce total server count, floor space, power and cooling. At least as important - and mentioned less often - is the fact that virtualisation encapsulates a machine as a set of files. Copyable, movable, restorable. Elasticity. Replication. High-availability: discussed elsewhere in this report, built upon this.

4.2 The layers of virtualisation

Two taxonomies are useful and they answer slightly different questions. The first classifies virtualisation by which resource is being abstracted (Zhang 2026b; Erl, Puttini & Mahmood 2013).

Table 5. Virtualisation classified by the resource abstracted.

Type	What is abstracted
Processor virtualisation	Enables a processor to be shared across multiple application instances. Each VM sees logical CPUs that the hypervisor schedules onto physical cores.
Memory virtualisation	Aggregates memory resources into a single pool and manages that memory on behalf of the multiple applications using it, mapping guest-physical addresses to host-physical ones.
Network virtualisation	Entails virtual IP management and segmentation — virtual switches, port groups, VLANs — so physical routers and switches become logical network fabrics.
Storage virtualisation	Provides a layer of abstraction for the physical storage of data, presenting physical arrays as one logical pool. Question 5 covers this in detail.
Power virtualisation	A physical UPS can be abstracted into a virtual UPS, pushing the same idea down into facility infrastructure.

The second classifies by where in the software stack the abstraction is inserted. This one explains why different technologies behave so differently on isolation and overhead:

- **Hardware or platform level.** The hypervisor virtualises the machine itself, and each guest runs its own kernel. Can be implemented using full virtualisation (with binary translation), paravirtualisation (where the guest is modified to call the hypervisor directly) (Barham et al. 2003), or with hardware assistance (VT-x or AMD-V). Has strongest isolation, but comes with highest per- instance overhead.
- **Operating-system level.** Virtually presents multiple isolated user- space instances, i.e. containers. Has very low overhead and fast start- up, but because every instance shares that one kernel isolation is weaker and the guest cannot run an OS different from the host.
- **Application level.** A managed runtime such as the JVM or the .NET CLR virtualises the execution environment for a single application, gaining portability across OSes without virtualising the machine at all.
- **Desktop and presentation level.** Virtual desktop infrastructure (VDI) and application publishing technologies separate the user interface from the physical machine doing the execution.

4.3 What is a hypervisor?

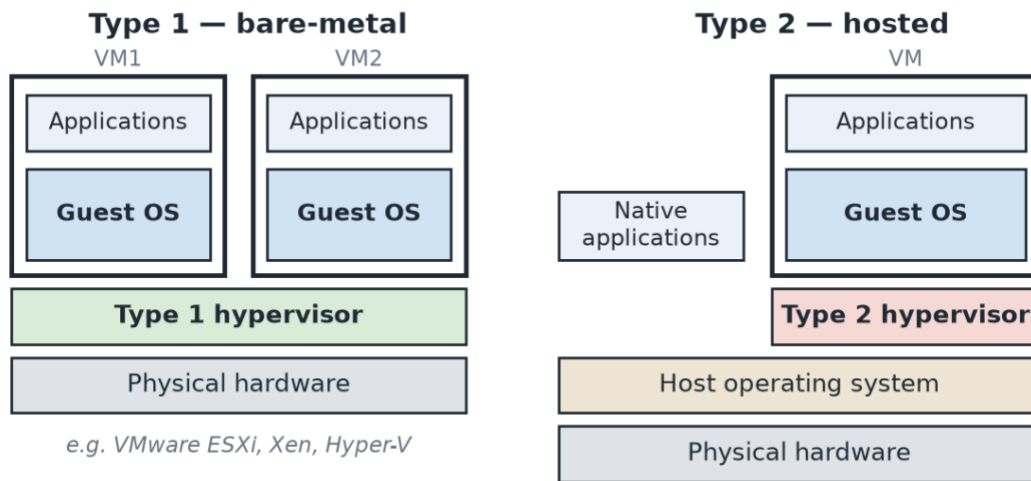


Figure 4. Type 1 (bare-metal) and Type 2 (hosted) hypervisors. A Type 1 hypervisor runs directly on hardware; a Type 2 hypervisor runs as an application above a host operating system. Adapted from Zhang (2026b).

Strictly speaking, a hypervisor or virtual machine monitor is the software layer that sits beneath the applications' operating systems (AKA guest OSs) and controls their use of CPU, memory, storage, etc. It enables isolation between several VMs hosted on the same physical machine and manage these VMs, allocating each VM with logical CPU and memory, logical storage blocks, and logical network resources (Zhang 2026b). The result is that it produces something known as a virtual machine or VM: an isolated environment containing application software and a guest operating system that acts like another discrete computer by virtualizing its hardware, virtual disks, and related metadata (Zhang 2026b).

Hypervisors come in two flavours. Install a Type 1 or bare-metal hypervisor such as VMware ESXi, Xen, Hyper-V directly onto the hardware with zero host operating system below it. Less OS in the I/O path means less overhead and a smaller TCB, which is why production data centres use Type 1 hypervisors almost exclusively. A Type 2 or hosted hypervisor like VMware Workstation or VirtualBox runs as a regular application atop some host operating system alongside all your other native applications. It's very convenient for PCs, training, and development purposes, but you pay for that convenience with another layer of scheduling and I/O.

If you look at the labs we're doing in the subject's own environment, it illustrates the difference rather nicely. For Week 2 lab, I installed ESXi on a VM with 2 vCPU, 4GB RAM, and 18 GB disk using local install type which doesn't require any operating system below it (Zhang 2026d). Remember that this ESXi installation is itself nested inside VMware Workstation, which is a Type 2 hypervisor. For that reason, the host has to advertise its CPU virtualisation extensions for the inner ESXi install to do anything at all. I kinda liked this happy coincidence. It clearly shows a Type 1 hypervisor relying directly on CPU support for virtualisation rather than depending on services exposed by some host OS, because it simply does not have a host OS to depend on.

4.4 What does a hypervisor do?

Four principal functions (Zhang 2026b), each of which unpacks into concrete mechanisms.

6. Creating and managing VMs. The hypervisor creates virtual hardware from configuration and an image and drives the machine through its life cycle: powering on and off, suspending, resuming, snapshotting, cloning. This is the step where magic turns a machine into a file and vice versa.
7. Allocating 'hardware resources' to VMs out of the virtualised pool of 'hardware' belonging to the physical server. This means in practice time-slicing physical cores between virtual CPUs; mapping guest-physical memory to host-physical memory through nested paging (Intel's EPT) or explicitly (shadow page tables where hardware cannot assist); reclaiming memory from guests via ballooning and page sharing; presenting storage and network devices either by emulating real hardware or through paravirtual drivers. Allocation is controlled according to policy, rather than being first-come-first-served: when running the Week 6 lab two resource pools with different share values and 500 MHz CPU limit results in visibly different completion times for the same CPU-bound script running in VMs on a single host (Zhang 20 26e). Resource allocation controlled...
8. Monitoring status of VMs. Information which the hypervisor is the only party in a position to see about what each VM actually consumes gets passed along to the Cloud OS information manager subsystem and accounting subsystem described in Question 3.
9. Participating in movement of VMs from one system to another. Live migration works by copying memory pages over to the destination host while the VM runs, re-copying pages that got changed during the copy, freezing the machine briefly to move over the last of its state and its CPU registers, then resuming execution on the other host (VMware 2011). VMware vMotion moves VMware virtual machines while they are live and running from one host to another without loss of service availability (Zhang 20 26c), which is exactly what you did in the Week 4 lab, migrating between ESXi01 and ESXi02 (Zhang 20 26f).

In fact, all four derive their names from the property the hypervisor is named after: it runs at a privilege level higher than any guest kernel, which allows it to trap privileged instructions and mediate them. That mediation is what allows the hypervisor its power. It's also why the hypervisor represents the most security-critical piece of software in the entire stack, because compromising it allows you to break the isolation of every VM layered above it simultaneously.

Question 5

5.1 What is storage virtualisation?

Storage virtualisation is:

"the process of presenting a logical view of the physical storage resources to a host computer system, treating all storage media (hard disk, optical disk, tape, etc.) in the enterprise as a single pool of storage" (Zhang 2026b).

Put more plainly, it is the storage case of the general abstraction from Question 4. Hosts address logical capacity and a mapping layer quietly translates that into physical extents on real media.

5.2 What does storage virtualisation do?

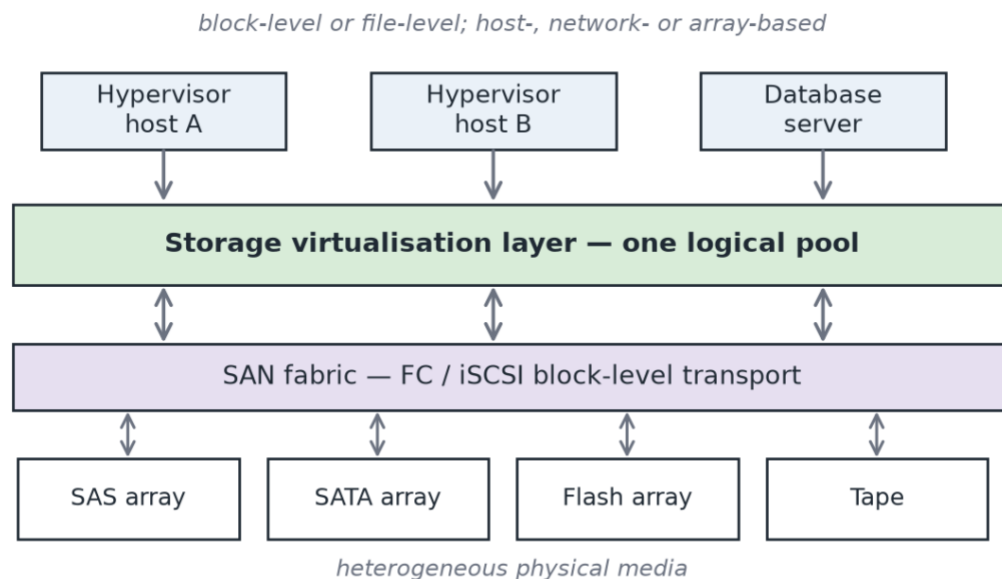


Figure 5. Storage virtualisation presents heterogeneous physical media to hosts as one logical pool. The layer can live in the host, in the network or in the array.

According to the above subject matter, virtualization falls into two categories along two axes (Zhang 2026b). With respect to granularity, there are block-level virtualization, which abstracts raw blocks and LUNs, and file-level virtualization, which abstracts a file system namespace. With respect to placement, there are three methods: network-based virtualization, where a mapping is performed by an appliance or switch placed in the fabric; host-based virtualization, where a volume manager or hypervisor performs the mapping; and array-based virtualization, where virtual volumes are presented by the storage controller itself.

The work it actually performs:

- **Pooling and abstraction.** Storage virtualization hides and controls complexity and heterogeneity in the physical storage subsystems so that a host is agnostic to vendor, protocol, or geometry of the device presenting storage to it (VMware 2009).
- **Presentation as local devices.** Within each VM, applications and guest OS see storage presented as SCSI disks connected to a virtual host bus adapter and those virtual SCSI disks are provisioned from datastore elements within the data centre (iTune) (VMware 2009). To the guest the disk it is using appears to be a local disk when in reality it is a file on a shared array.
- **Capacity efficiency.** Features like thin provisioning (allocates capacity on write, not create) and dedupe/compression (reduce the footprint of similar data) are implemented in software.
- **Non-disruptive data mobility.** Because the host accesses a logical volume, the physical mapping can change while I/O continues uninterrupted. Data can move between arrays or tiers without downtime.
- **Data services.** Functions like snapshots, clones, replication and automated tiering are implemented once, at the virtualization layer then applied consistently across whatever blend of back-end hardware may be present (Tate et al. 2018).

Block access (over iSCSI, Fibre Channel or FCoE LUNs), file access (over POSIX APIs like NFS and CIFS), object access (using HTTP CRUD interfaces) and dataset access (via DBMS API) all co-exist above the same level of abstraction (Zhang 2026c). In VMware terminology, Datastores formatted with VMFS or exposed over NFS exist above DAS, SCSI, FC SAN, iSCSI and NAS hardware, while the VMs presented with those Datastores never know the difference (Zhang 2026b).

5.3 When should storage virtualisation be used?

The lectures define the technology but do not say when to reach for it, so this section draws on vendor documentation and industry practice. The case is strong when any of the following holds.

- Many hosts require access to the same data. Live migration/high-availability clustering/fault tolerance all need the destination host to access the disks associated with the source VM. As such shared, virtualised storage becomes a prerequisite, not an optimisation (VMware 2011).
- The environment is mixed. Where mixed vendor or generation arrays exist side by side a virtualisation layer provides one management model, one set of data services and one pool of capacity rather than multiple islands (Tate et al. 2018).
- Usage is low and/or fragmented. Capacity trapped on per-server islands is wasted capacity. Combine pooling with thin provisioning and it is useful capacity again. This is essentially server consolidation applied at the disk level.
- Hardware refresh must be done without outage. Organizations that run continuous-availability systems need to migrate data between arrays with applications running. The abstraction layer simplifies this to routine maintenance rather than a harrowing experience.
- Multi-tenancy or chargeback requirements exist. Dividing a pool into logical volumes per tenant, with quotas and metered usage, is far superior to physically allocating spindles to groups.

This isn't always correct either. If you've got a latency sensitive workload that exists on a single host then direct attached storage avoids a network hop and a translation layer. In small estates the cost of licensing, increased complexity and additional failure domain can far outweigh the benefits you gain, and an in-band virtualisation appliance that hasn't been highly available-ified itself is a single point of failure for all hosts piled behind it. So really the decision is dependent on requirements around sharing, heterogeneity of your estate and your availability requirements rather than how much capacity you are dealing with..

5.4 What is a SAN?

A Storage Area Network (SAN) is a high-speed dedicated network that connects servers to specialized/shared consolidated block-level storage devices (Storage devices) over a SAN switch fabric. Most commonly, they use Fibre Channel optics or transport over IP using iSCSI (Zhang 20 26b; Tate et al. 2018). SAN is one of three common storage topologies. Defining SAN in opposition to the other two will get you faster than a paragraph.

Table 6. DAS, NAS and SAN compared (after Zhang 2026b).

	DAS	NAS	SAN
Connection	Directly attached to one server via a host bus adapter	Attached to the general-purpose LAN via an Ethernet switch	Attached via a dedicated storage fabric (FC or iSCSI)
Access granularity	Block	File (NFS, CIFS)	Block (LUNs)
Sharing	None, captive to its host	File-level sharing among many clients	The same LUNs presented to multiple hosts
Typical use	Local boot, scratch space, small servers	File shares, home directories, some VM datastores	Databases, mail servers, hypervisor datastores

5.5 What does a SAN do?

Three things a directly attached disk cannot.

10. It abstracts storage away from the server. Storage capacity resides in shared pools, which are allocated to hosts on demand. So database servers, mail servers, and pools of virtual machines can all utilize the same storage infrastructure (Zhang 2026b). Increasing storage capacity or reallocating it to different servers doesn't require you to pop open a server chassis.
11. It transports block I/ O that behaves like a local disk. Network storage volumes appear to the OS as standard SCSI disks, so applications that require raw block access can operate unmodified on remote LUNs without being confined to shared folders. If storage traffic is load balanced over multiple links (Multipathing) and ironclad against hardware failures (via redundant fabrics), guests can achieve even higher levels of performance.
12. It facilitates shared storage access across the cluster. That's what makes a storage area network so critical to virtualization and why VMs have been able to enjoy features like live migration and high availability. Because multiple hypervisors can access the same datastore simultaneously, a virtual machine's files are available to the rest of the cluster even as it runs (Tate et al. 2018).

Point number three became real for me during Week 3 lab. I created an iSCSI target at IP address 172.20.20.50 as a dynamic target on ESXi01's software iSCSI adapter, then repeated the process on ESXi02. The exact same FreeNAS iSCSI device showed up on both hosts, and I was able to create a VMFS datastore accessible by both hosts on that target (Zhang 2026g). I realized why the following week: the Week 4 lab begins by forcing you to confirm that both hosts can still connect to last week's iSCSI target before it will allow you to migrate a virtual machine between them (Zhang 2026f). Storage virtualisation over a SAN, then, is not just clean capacity management. Before a running workload can migrate at all, and therefore before most of the elasticity and availability behaviour expected by the rest of the cloud model, that SAN storage must exist.

Question 6

Creating an IaaS platform is significantly more difficult than collecting components that provide pieces of the capabilities required. This is because the platform must provide isolation, elasticity, and measured service simultaneously and continuously. Seven challenges are discussed below with possible mitigations. The first five challenges are those returned to most frequently in the literature (Zhang, Cheng & Boutaba 2010; Buyya et al. 2009).

Table 7. Seven challenges in building an IaaS platform and their mitigations.

Challenge	Why it is difficult	How to deal with it
1. Multi-tenant isolation and security	Tenants share physical hosts, fabrics and arrays. Ristenpart et al. (2009) showed an attacker can deliberately place a VM on the same physical machine as a target and then extract information through side channels. The hypervisor is also a single point of total compromise.	Defence in depth: encryption at rest and in transit, hashing and digital signatures for integrity, PKI, identity and access management, single sign-on, cloud-based security groups and hardened virtual server images (Zhang 2026c). Add per-tenant logical network perimeters, disciplined hypervisor patching, and dedicated hosts for workloads that genuinely cannot tolerate co-residency.
2. Resource scheduling and over-subscription	Selling more virtual capacity than physically exists is what makes the economics work, but getting the ratio wrong causes contention and SLA breaches. Scheduling has to happen at three levels at once — hypervisor, cloud and federation (Zhang 2026a).	Use shares, reservations and limits so priority is stated rather than emergent. Enable dynamic load balancing across the cluster. Apply admission control so placement is refused up front rather than accepted and then violated. Keep capacity headroom sized to the failure domain, not to the average.
3. Elasticity under bursty demand	Provisioning has to track demand within minutes. Scale too slowly and requests are lost; scale too eagerly and you waste capacity, spend money and set the system oscillating.	Implement the dynamic scalability and workload distribution architectures: an automated scaling listener, resource replication from stored images, load balancing across the replicas (Zhang 2026c). Add hysteresis and cool-down periods to the scaling policy, and keep pre-warmed pools for anything whose start-up time is longer than the burst itself.
4. Availability and fault tolerance	Commodity hardware fails as a matter of routine, yet the platform has to present availability none of its components individually has. Correlated failures — a rack, a fabric, a power feed — defeat naive redundancy entirely.	Cluster hosts with high availability so VMs restart automatically on survivors, and use fault tolerance where a restart is not acceptable (Zhang 2026c). Require shared storage, redundant fabrics and multipathing. Use anti-affinity rules to keep replicas apart. Size N+1 against the largest correlated failure domain rather than the smallest.

5. Performance isolation, the 'noisy neighbour'	Even with capacity correctly provisioned, one tenant's I/O or network burst degrades another tenant's latency. Averages keep looking healthy while individual tenants experience unusable tail latency.	Apply storage and network I/O control with per-VM shares and limits. Separate management, migration and tenant traffic onto different port groups or physical adapters. Monitor percentile latency instead of means, and put latency-sensitive workloads on dedicated tiers.
6. SLA assurance, metering and billing accuracy	Measured service is an essential characteristic of the model (Mell & Grance 2011), which makes metering errors immediately commercial. Availability has to be measured continuously, not asserted retrospectively.	Deploy cloud usage monitors in all three forms — monitoring agent, polling agent, resource agent — feeding a log database, with an SLA management system and a billing management system built on top (Zhang 2026c). Reconcile meters against an independent source, and publish consumption to tenants so discrepancies surface early rather than at invoice time.
7. Interoperability and vendor lock-in	Most cloud products and providers offer their own APIs rather than a common one (Zhang 2026a), so images, network definitions and automation all become provider-specific. Opara-Martins, Sahandi and Tian (2016) identify lock-in as a leading barrier to adoption.	Prefer portable formats such as OVF for images. Keep infrastructure definitions in provider-neutral infrastructure-as-code wherever it is practical. Use a federation manager to abstract remote clouds (Zhang 2026a). Validate the exit path by rehearsing a migration occasionally, rather than assuming one would work.

The labs simulate only a couple of these, and smaller, than what I was used to. I didn't anticipate it being as educational as it was. Connectivity from host to ESXi relies on every virtual NIC being on the correct vmnet segment, and every host having the correct static address, and both Week 2 and Week 3 start with troubleshooting documentation for lost connectivity because nothing else can be done until it is remediated(Zhang 2026d, 2026g). Week 4 starts with ensuring licences are valid for vCenter and its hosts(Zhang 2026f). Neither issue is conceptually complex. Both illustrate the same point as the table above. Within an IaaS platform, a single mis-configured item in the control plane denies users access to resources that are otherwise completely healthy. Availability is impacted by the management pathway at least as much as the physical hardware itself.

Question 7

7.1 Network as a Service

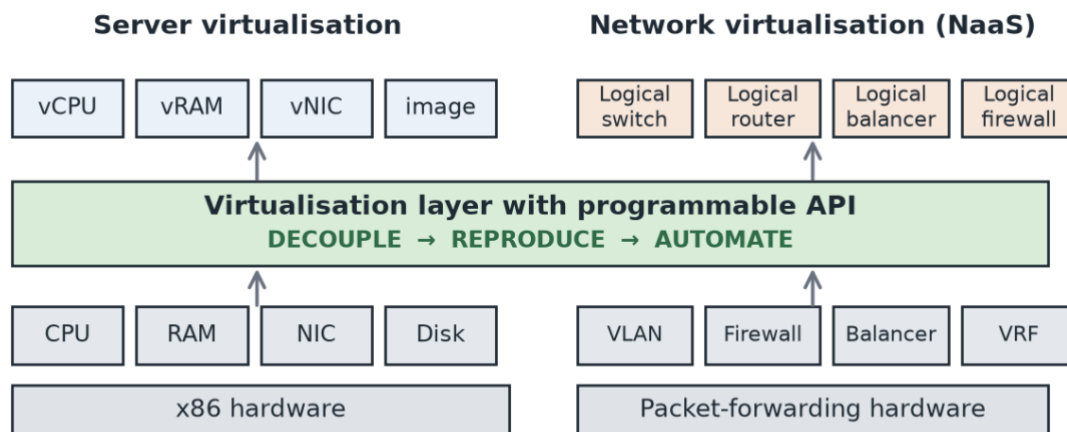


Figure 6. Network virtualisation applies to packet forwarding the same decoupling that server virtualisation applied to x86 hardware, reproducing switching, routing, load balancing and firewalling as software services above a programmable API. Adapted from Zhang (2026b) and VMware (2013).

Network as a Service refers to a cloud service category in which “the capability provided to the customer is transport connectivity and related network capability” (ISO/IEC 2014). Rather than purchasing, wiring, and configuring switches, routers, load balancers, and firewalls, the consumer specifies network functions and connectivity as needed, and pay-as-you-go bills for what they use. It’s fundamentally nothing more than IaaS applied to networking rather than compute.

How can this be done? Network virtualization. VMware (2013) nicely summarizes it as “network virtualization does for the network what server virtualization did for the compute platform”, in three steps: dissociate the logical function from the physical hardware which implements it, instantiate that function in software, and automate its provisioning using an API.

Server virtualization of x86 platforms gave rise to the vCPU, vRAM, vNIC and machine image. Network virtualization of packet-forwarding devices produces the logical switch, logical router, logical load balancer, and logical firewall (Zhang 2026b). Taken together, the upshot is that you can programmatically create, clone, or destroy an entire multi-tier topology, policy included, in the amount of time it takes you to run a script.

You can already see these building blocks at work in the platform we’ve been studying in this course. A vSphere standard switch exposes port groups, which bundle together multiple ports under a common configuration profile and provide virtual machines with a persistent anchor point from which to reach logically-segregated networks, physical adapters beneath (VMware 2011; Zhang 2026b). A VM connects to a port group by name, not to a physical adapter, so it retains its network identity when it migrates. Same pattern, just one layer down.

Extend NaaS to cloud scale, and it stretches beyond the data center. Duan, Yan, and Vasilakos (2012) review how, in practice today meaning virtual private cloud networks with tenant-specific address space, bandwidth, and VPN connectivity on demand; load balancing and firewalling offered as metered services; and provider-managed dedicated interconnection between an enterprise and cloud provider. Erl's concept of a logical network perimeter – logically isolating a network segment from the rest of a communications network – is sitting below all of them (Zhang 20;26c). Fundamentally, NaaS is better thought of as something alongside IaaS in the service category taxonomy, rather than as a layer above or below it. NaaS delivers network capability with the same self-service, metering properties that IaaS provides compute capability.

7.2 Cloud ecosystems

A cloud ecosystem is the collaborating group of stakeholders, services and business relationships that associate with a cloud platform to provide value to a consumer. The term exists because a single entity does not offer a complete solution. The elements that create what the consumer ultimately sees is formed by a chain of specialised vendors. Each segment of that chain can come from a different organization.

This subject follows the NIST reference architecture that lists five major actors (Liu et al. 2011), using very similar language (Zhang 20 26b).

Table 8. Actors in a cloud ecosystem (after Liu et al. 2011).

Actor	Role in the ecosystem
Cloud consumer	An organisation or person holding a formal contract with a provider to use IT resources the provider makes available. The party actually getting business value out of the arrangement.
Cloud provider	The company or third party making cloud services available, responsible for service deployment, orchestration, management, security and privacy.
Cloud carrier	Provides connectivity and transport between consumers and providers. Without a carrier the 'broad network access' characteristic cannot be met at all.
Cloud broker	Manages the use, performance and delivery of cloud services and negotiates relationships between providers and consumers, adding value through intermediation, aggregation or arbitrage.
Cloud auditor	Conducts independent assessment of services, operations, performance and security. This is how a provider's claims become verifiable to a consumer who has no way of inspecting the infrastructure.

The other two roles from our subject terminology that help define this picture are the cloud service owner (the person or organisation who legally owns a cloud service) and the cloud resource administrator (the party responsible for administering cloud-based IT resources(Zhang 20 26b)). They warrant inclusion here because the ownership, operation, and consumption responsibilities often reside with three different parties. For example, a SaaS provider may own a service that runs on another company's IaaS and is administered on a day-to-day basis by a third company's managed-service team. Beyond the actors, there are three dynamics at play in an ecosystem that's functioning. These also explain why providers go through so much effort to build one:

- Complementarity and marketplaces. Third parties enhance the platform and sell to platform users via its marketplace. This could be an ISV, a consultancy, or a managed-service partner. Each new capability increases the platform's value without marginal cost to the platform provider.
- Standards and interoperability. Common vocabularies and interfaces—including the ISO/IEC (2014) overview and vocabulary, among others—allow ecosystem players to aggregate services from multiple providers. This is a prerequisite for brokerage and for most types of hybrid architecture.
- Network effects, and its dark side. The more complements in an ecosystem, the more consumers will join it, which in turn will attract more complements. The same force that creates the network effect can create switching costs (see Question 6). That is why lock-in is an ecosystem-level challenge and not merely an API-level one (Opara-Martins, Sahandi & Tian 2016).

Question 8

8.1 What is a data center?

A data centre is a facility used to house an organisation's computer systems and associated components, such as telecommunications and storage systems. Also included are the power distribution and cooling systems needed to run them, as well as security systems and mechanisms. Rather than being strictly defined by dimensions, a data centre is better understood by its functionality. It houses where many IT resources are placed centrally, to allow them to be managed, secured and shared more effectively than if distributed elsewhere.

A data centre differs from a large server room in two main areas. Firstly, it has engineered redundancy of its infrastructure. The Uptime Institute's (2018) Tier Classification System rates data centres from Tier I upwards according to how redundant their subsystems are — from basic capacity with single distribution paths with no redundancy at Tier I to fault tolerant, with multiple actively redundant distribution paths able to undergo maintenance while remaining operational at Tier IV. TIA-942 defines equivalent criteria for cabling, spatial design and infrastructure requirements (TIA 2017). Secondly, the entire facility is designed and treated as one unified machine. Barroso, Clidaras and Hölzle (2013) argue that when considering data centres at cloud scale, the building should be thought of as the computer. This 'warehouse-scale computer' is designed with the efficiency of the whole system in mind — often measured in power usage effectiveness — rather than focusing on the performance of individual servers. Cloud computing requires data centres. Their resources need to be pooled together, which requires them to be physically close together and densely interconnected. Rapid elasticity requires spare capacity to be available at a moment's notice. Measured service requires the metering technology described in Question 3. Perhaps unsurprisingly then, you'll find 'data centre technology' is one of the technology innovations listed as part of a cloud service provider's infrastructure stack (Zhang 2026c).

8.2 Basic components of a data centre

The standard grouping splits the facility four ways (Zhang 2026c; Erl, Puttini & Mahmood 2013).

Table 9. Basic components of a data centre grouped by category (after Zhang 2026c).

Category	Components
Facilities	Housing, racks, cabling, power supplies and the environmental control station: heating, ventilation, air conditioning, fire protection and other subsystems. In practice this also takes in the power chain (utility feed, transformers, UPS, standby generators, power distribution units), hot- and cold-aisle containment, and physical access control.
Computing hardware	Servers and blade servers, typically commodity x86 machines grouped into clusters and running a Type 1 hypervisor.
Storage hardware	Hard disk arrays, Storage Area Network (SAN) and Network-Attached Storage (NAS) systems, with the virtualisation layer from Question 5 presenting them as pooled capacity.
Network hardware	Carrier and external network interconnection, web-tier load balancing and acceleration, LAN fabric, SAN fabric and NAS gateways.

The schematic above is a simplified depiction of the physical topology of an example VMware infrastructure data centre (VMware 2011; Zhang 2026b): there's a vCenter Server providing centralised management that vSphere and web clients connect to, there are groups of physical servers, each running ESX or ESXi and hosting virtual machines; and there's a fibre channel switch fabric or IP network connecting those hosts to fibre channel, iSCSI and NAS storage arrays. Every element in the diagram has a direct counterpart in the real world. Server groups are the computing hardware; the arrays are the storage hardware; the fabric is the network hardware; and the racks/powers/cooling containing all of them are the facilities themselves.

One item's easy to leave off that list, but which absolutely belongs on it: the management and monitoring systems themselves. Whether that's specialised data centre infrastructure management software, environmental sensors, or even the cloud usage monitors from Question 3 above, these are what make running a facility of this size possible, and you'll know quickly when they're missing because your hardware will start failing..

References

Harvard (UTS) referencing style.

- Armbrust, M., Fox, A., Griffith, R., Joseph, A.D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I. & Zaharia, M. 2010, 'A view of cloud computing', *Communications of the ACM*, vol. 53, no. 4, pp. 50–58.
- Barham, P., Dragovic, B., Fraser, K., Hand, S., Harris, T., Ho, A., Neugebauer, R., Pratt, I. & Warfield, A. 2003, 'Xen and the art of virtualization', *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, ACM, New York, pp. 164–177.
- Barroso, L.A., Clidaras, J. & Hölzle, U. 2013, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*, 2nd edn, Morgan & Claypool, San Rafael, CA.
- Buyya, R., Yeo, C.S., Venugopal, S., Broberg, J. & Brandic, I. 2009, 'Cloud computing and emerging IT platforms: vision, hype, and reality for delivering computing as the 5th utility', *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616.
- Duan, Q., Yan, Y. & Vasilakos, A.V. 2012, 'A survey on service-oriented network virtualization toward convergence of networking and cloud computing', *IEEE Transactions on Network and Service Management*, vol. 9, no. 4, pp. 373–392.
- Erl, T., Puttini, R. & Mahmood, Z. 2013, *Cloud Computing: Concepts, Technology & Architecture*, Prentice Hall, Upper Saddle River, NJ.
- ISO/IEC 2014, *ISO/IEC 17788:2014 Information technology — Cloud computing — Overview and vocabulary*, International Organization for Standardization, Geneva.
- Liu, F., Tong, J., Mao, J., Bohn, R., Messina, J., Badger, L. & Leaf, D. 2011, *NIST Cloud Computing Reference Architecture*, NIST Special Publication 500-292, National Institute of Standards and Technology, Gaithersburg, MD.
- Marston, S., Li, Z., Bandyopadhyay, S., Zhang, J. & Ghalsasi, A. 2011, 'Cloud computing — the business perspective', *Decision Support Systems*, vol. 51, no. 1, pp. 176–189.
- Mell, P. & Grance, T. 2011, *The NIST Definition of Cloud Computing*, NIST Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg, MD.
- Opara-Martins, J., Sahandi, R. & Tian, F. 2016, 'Critical analysis of vendor lock-in and its impact on cloud computing migration: a business perspective', *Journal of Cloud Computing*, vol. 5, no. 4, pp. 1–18.
- Popek, G.J. & Goldberg, R.P. 1974, 'Formal requirements for virtualizable third generation architectures', *Communications of the ACM*, vol. 17, no. 7, pp. 412–421.
- Ristenpart, T., Tromer, E., Shacham, H. & Savage, S. 2009, 'Hey, you, get off of my cloud: exploring information leakage in third-party compute clouds', *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS '09)*, ACM, New York, pp. 199–212.
- Tate, J., Beck, P., Ibarra, H.H., Kumaravel, S. & Miklas, L. 2018, *Introduction to Storage Area Networks*, IBM Redbooks SG24-5470-08, IBM Corporation, Poughkeepsie, NY.
- Telecommunications Industry Association (TIA) 2017, *TIA-942-B Telecommunications Infrastructure Standard for Data Centers*, TIA, Arlington, VA.
- Uptime Institute 2018, *Tier Classification System*, Uptime Institute, New York.
- VMware 2009, *VMware vSphere Basics: Storage Architecture*, VMware Inc., Palo Alto, CA, viewed 27 August 2026, <<http://pubs.vmware.com/vi3/intro/wwhelp/wwhimpl/common/html/wwhelp.htm>>.

- VMware 2011, VMware vSphere Basics: Network Architecture and Physical Topology of the VMware Infrastructure Data Center, VMware Inc., Palo Alto, CA, viewed 27 August 2026, <<https://pubs.vmware.com/vsphere-50/>>.
- VMware 2013, VMware NSX Network Virtualization Platform, white paper, VMware Inc., Palo Alto, CA, viewed 27 August 2026, <<https://www.vmware.com/content/dam/digitalmarketing/vmware/en/pdf/whitepaper/products/nsx/vmware-nsx-network-virtualization-platform-white-paper.pdf>>.
- Zhang, H. 2026a, Week 1: An Introduction, 41891/42891 Cloud Computing Infrastructures lecture notes, University of Technology Sydney, Sydney.
- Zhang, H. 2026b, Week 2: Cloud Infrastructure Design — Virtualization Technologies, 41891/42891 Cloud Computing Infrastructures lecture notes, University of Technology Sydney, Sydney.
- Zhang, H. 2026c, Week 3: Infrastructure/Management Mechanisms and Architectures, 41891/42891 Cloud Computing Infrastructures lecture notes, University of Technology Sydney, Sydney.
- Zhang, H. 2026d, Week 2 Lab: VMware ESXi, Build VMs, 41891/42891 Cloud Computing Infrastructures lab guide, University of Technology Sydney, Sydney.
- Zhang, H. 2026e, Week 6 Lab: Resources, 41891/42891 Cloud Computing Infrastructures lab guide, University of Technology Sydney, Sydney.
- Zhang, H. 2026f, Week 4 Lab: vCenter Server and VM Migration, 41891/42891 Cloud Computing Infrastructures lab guide, University of Technology Sydney, Sydney.
- Zhang, H. 2026g, Week 3 Lab: iSCSI (VMFS) on ESXi, 41891/42891 Cloud Computing Infrastructures lab guide, University of Technology Sydney, Sydney.
- Zhang, Q., Cheng, L. & Boutaba, R. 2010, 'Cloud computing: state-of-the-art and research challenges', Journal of Internet Services and Applications, vol. 1, no. 1, pp. 7–18.